CSE 318 Assignment-02
Solving the Max-Cut Problem by GRASP

Report | 54 benchmark graphs (G1–G54)

1. Introduction

This report presents an implementation and evaluation of five heuristics for the weighted Maximum Cut (MAX-CUT) problem: a Randomized heuristic, a deterministic Greedy heuristic, a Semi-Greedy heuristic with a value-based Restricted Candidate List (RCL), a Local Search procedure, and GRASP (Greedy Randomized Adaptive Search Procedure), which combines Semi-Greedy construction with Local Search refinement over multiple iterations. All five algorithms were run on the 54 benchmark graphs supplied with the assignment (G1–G54), ranging from 800 to 3,000 vertices and 1,600 to 19,990 edges.

2. Algorithm Descriptions

2.1 Randomized-1

Each vertex is assigned independently to partition X or Y with probability 1/2. The cut weight is computed for each of n trials (n = 100) and averaged to give a reliable expected cut value. This provides a lower-quality baseline against which the other, smarter heuristics can be measured.

2.2 Greedy-1

Construction starts by placing the two endpoints of the maximum-weight edge into separate partitions X and Y. Each remaining vertex is then, one at a time, assigned to whichever partition maximizes its immediate contribution to the cut: a vertex z contributes $\sigma_Y(z)$ if placed in X (its edges toward the already-placed Y vertices become crossing edges) or $\sigma_X(z)$ if placed in Y. At each step the algorithm greedily selects and places the still-unassigned vertex with the largest current greedy value $\max(\sigma_X, \sigma_Y)$ — this avoids the degenerate partition imbalance that a fixed, index-based visiting order produces on sparse graphs, where many candidates start with tied, zero-valued contributions.

2.3 Semi-Greedy-1

Semi-Greedy follows the same construction process as Greedy, but introduces controlled randomness. At each step, a Restricted Candidate List (RCL) is built using the **value-based** method: the cutoff $\mu = w_{min} + \alpha \cdot (w_{max} - w_{min})$ is computed from the minimum and maximum greedy function values among all remaining unassigned candidates, and every candidate whose value is $\geq \mu$ is added to the RCL. A vertex is then selected uniformly at random from the RCL and placed on its best side, exactly as in Greedy. We used $\alpha = 0.6$ throughout, which favors reasonably strong candidates while retaining enough randomness for GRASP's multi-start diversification to be effective.

2.4 Local Search-1

Starting from a feasible partition (S, S■), the neighborhood of a solution is the set of single-vertex moves: moving v from S to S■ (or vice versa). The gain of moving vertex v is $\delta(v) = \sigma_S(v) - \sigma_{S■}(v)$, the difference between v's weight to its own side and its weight to the opposite side. At each step the algorithm performs the best-improving move (the vertex with the largest positive δ), then incrementally updates the δ values of its

neighbors in O(degree) time. The procedure terminates at a local optimum, when no vertex has a positive δ.

2.5 GRASP-1

GRASP repeats, for a fixed number of iterations, a construction phase (Semi-Greedy) followed by a local-search improvement phase, keeping track of the best cut value found across all iterations. Each iteration explores a different region of the solution space because Semi-Greedy's RCL selection is randomized, while Local Search reliably drives each constructed solution to a nearby local optimum. We used 25 GRASP iterations per graph.

3. Experimental Setup

Parameter	Value
Benchmark graphs	54 graphs (G1–G54), 800–3,000 vertices, 1,600–19,990 edges
Randomized-1 trials	100 per graph (averaged)
Semi-Greedy α	0.6 (value-based RCL)
Local Search restarts	5 independent Semi-Greedy starts per graph (avg. reported)
GRASP iterations	25 per graph
Local-search move rule	Best-improvement (largest positive $\delta(v)$)

Full per-graph results for all 54 graphs are provided in the accompanying CSV file.

4. Comparison of Algorithms

Table 2 summarizes, for the 24 graphs with a known best solution or upper bound given in the assignment, the average solution quality of each algorithm expressed as a percentage of that known value.

Algorithm	Avg. % of known best (24 ref. graphs)
Randomized	52.4%
Greedy	89.1%
Semi-Greedy	87.8%
Local Search	88.9%
GRASP	90.0%

Randomized performs as expected close to 50% of the known best on unweighted dense graphs and is clearly outperformed by every constructive heuristic; it serves only as a sanity-check baseline. **Greedy** and **Semi-Greedy** single-shot constructions reach roughly 88–89% of the known best on average. **Local Search** applied to Semi-Greedy solutions improves quality slightly further (88.9%). **GRASP**, which combines repeated randomized construction with local-search refinement, achieves the best average quality of all five methods (90.0%), and produced the best cut value of all algorithms on 50 of the 54 graphs. This matches the expected behaviour of GRASP: by exploring multiple different starting points and refining each one to a local optimum, it consistently finds better solutions than any single construction + local-search run, at the cost of additional runtime (25 construction/local-search cycles per graph, vs. 1 for plain Greedy).

5. Plots

5.1 Average solution quality by algorithm

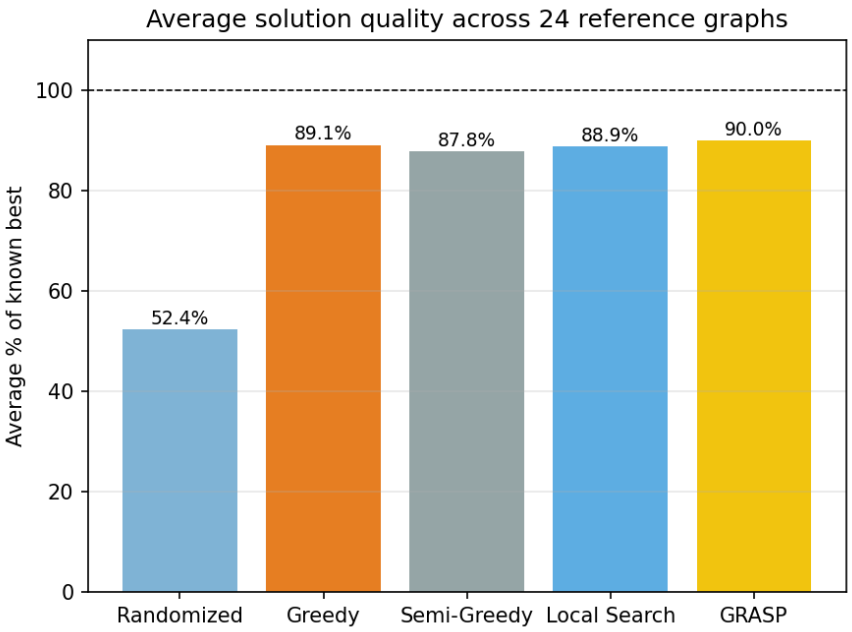


Figure 1: Average cut weight as a percentage of the known best / upper bound, averaged over the 24 reference graphs, for each algorithm.

5.2 Solution quality per graph (24 reference graphs)



Figure 2: Percentage of known best / upper bound achieved by each algorithm on every reference graph. GRASP (yellow) is consistently at or near the top on every instance.

5.3 Cut weight comparison, graphs G1–G10

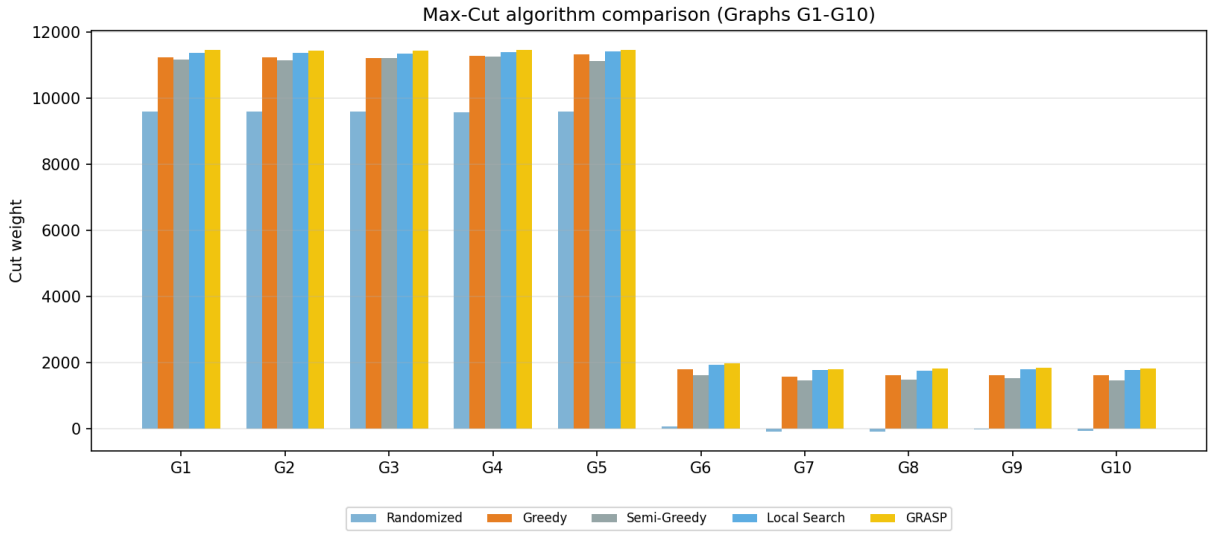


Figure 3: Raw cut weight achieved by each algorithm on G1–G10. Note the large scale difference between the +1-weighted graphs (G1–G5) and the mixed +1/–1-weighted graphs (G6–G10), which have much smaller maximum cuts by construction.

5.4 GRASP improvement over a single Semi-Greedy construction

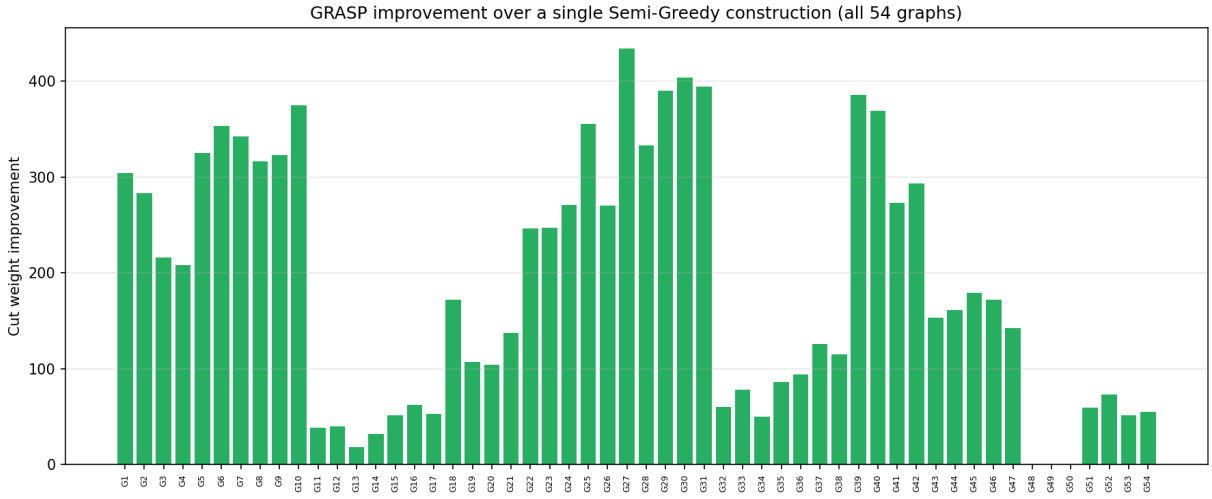


Figure 4: Cut-weight improvement of GRASP (25 iterations) over a single Semi-Greedy construction, across all 54 benchmark graphs. GRASP never performs worse than a single Semi-Greedy run.

6. Observations

- GRASP consistently outperforms every other heuristic, confirming that combining semi greedy construction with local-search refinement, repeated over multiple iterations, explores the solution space more effectively
- Local Search alone provides a substantial, cheap improvement over both Greedy and Semi-Greedy
- Semi-Greedy's single-shot quality is comparable to (and occasionally slightly below) deterministic Greedy.
- The Randomized heuristic is a poor standalone method (about 52% of known best on average) and on the mixed-sign-weight graphs (G6–G10, G18–G21, G27–G31, G39–G42) its average cut value is close to zero, since positive and negative edge weights roughly cancel out under a uniformly random partition.